

TOPIC 01: OOPS IN JS & OBJECT PROTOTYPES

Purpose of OOPs:

Object-Oriented Programming provides an architectural blueprint to structure codebase, prevent redundancy, and model real-world entities.

The 5 Core Pillars in JavaScript:

1. PROTOTYPES

2. NEW OPERATOR

3. CONSTRUCTORS

4. ES6 CLASSES

5. INHERITANCE (EXTENDS, SUPER)

Object Prototypes:

Prototypes are the core mechanism by which JavaScript objects **inherit properties and methods** from one another. It functions as a shared template object from which instances borrow functionality without duplicating memory copies.

- `arr.__proto__` → Direct pointer reference to `Array.prototype`.
- `String.prototype` → Stores built-in string methods (`trim`, `toUpperCase`, `slice`).

TOPIC 02: FACTORY FUNCTIONS & THEIR LIMITATION

What is a Factory Function?

A plain function that instantiates and returns a brand-new object literal:

```
function personMaker(name, age) { return { name: name, age: age, talk() { console.log(`Hi, I am ${this.name}`); } } }; let p1 = personMaker("Adam", 25); let p2 = personMaker("Eve", 25); console.log(p1.talk === p2.talk); // Output: FALSE (High Memory Overhead)
```



Drawback: Every newly created object stores a separate redundant copy of the `talk()` method! The optimal solution is **Constructor + Prototype**.

TOPIC 03: CONSTRUCTOR FUNCTIONS & 'NEW' OPERATOR

The 'new' Operator:

The new operator instantiates an object instance from a constructor function.

4 Internal Steps Executed by 'new':

1. CREATES NEW EMPTY OBJECT {}



2. BINDS 'THIS' TO NEW OBJECT



4. RETURNS THE OBJECT



3. LINKS __PROTO__ TO PROTOTYPE

Constructor with Shared Prototype:

```
// Constructor function (Convention: capitalized) function Person(name, age) {  
  this.name = name; this.age = age; } // Define method on prototype (Single  
shared instance in memory) Person.prototype.talk = function() {  
  console.log(`Hi, my name is ${this.name}`); }; let p1 = new Person("Adam",  
25); let p2 = new Person("Eve", 25); console.log(p1.talk === p2.talk); //  
Output: TRUE ✅ (Shared prototype reference!)
```



Key Takeaway: Attaching methods to prototype ensures that millions of objects reference a single function definition in memory!

TOPIC 04: ES6 CLASSES (MODERN SYNTAX)

What is an ES6 Class?

ES6 classes provide a cleaner, elegant **Syntactic Sugar** over JavaScript's existing prototype-based inheritance model.

```
class Person { constructor(name, age) { this.name = name; this.age = age; }  
talk() { console.log(`Hello, I am ${this.name}`); } } const p1 = new  
Person("Rahul", 21); p1.talk();
```

TOPIC 05: INHERITANCE (EXTENDS & SUPER)

Concept of Inheritance:

Enables a child/derived class to inherit attributes and methods from a parent/base class, eliminating duplicate logic.

- extends → Establishes the inheritance relationship between child and parent.
- super(args) → Invokes the constructor of the parent class to initialize inherited state.

```
// Parent Base Class class Person { constructor(name, age) { this.name = name;  
this.age = age; } } // Child Derived Class class Student extends Person {  
constructor(name, age, marks) { super(name, age); // Calls Person constructor  
this.marks = marks; } } const s1 = new Student("Aman", 20, 95);  
console.log(s1.name, s1.marks); // Aman 95
```



Mandatory Rule: In a derived class constructor, you must invoke `super()` before accessing the `this` keyword!

